

SGT UNIVERSITY

School of Engineering and Technology

Department of CSE

DATABASE MANAGEMENT SYSTEMS LABORATORY

Experiment 5

Aggregate Functions and Nested Queries

Field	Details
Student	Harshit Khemani
Roll No.	241302081
Programme	B.Tech CSE (AI/ML)
Section	Section - C
Lab Date	30 September 2026
Dataset	employee (16 rows; CSE, Mechanical, ECE, Civil)
Methods	SUM, AVG, MAX, MIN, COUNT, GROUP BY, HAVING, subqueries

1. Aim

To create an employee table, insert 16 rows across four departments, and answer the lab questions with aggregate functions (SUM, AVG, MAX, MIN, COUNT), GROUP BY / HAVING, and nested queries (a scalar subquery, IN / NOT IN, EXISTS, and correlated subqueries, including the second-highest salary).

2. Theory

2.1 Aggregate functions

An aggregate function computes one value from many rows. COUNT(*) counts rows. SUM(salary) adds the salary column. AVG(salary) is the arithmetic mean. MAX and MIN return the largest and smallest values. Without GROUP BY, each aggregate is taken over the whole table and the query returns one row. SQLite prints this table's average as 56125.0 because AVG returns a real number even when the mean is a whole number of rupees.

2.2 GROUP BY

GROUP BY department makes one group per distinct department, then evaluates the aggregates inside each group. Every column in the SELECT list must be either grouped or aggregated. Question 7 therefore returns four rows (CSE, Civil, ECE, Mechanical), each with its own count, total, average, highest salary and lowest salary.

2.3 HAVING

WHERE filters rows before grouping. HAVING filters groups after the aggregates are computed, so it can test AVG(salary) or COUNT(*). WHERE AVG(salary) > 55000 is illegal. Question 8 keeps departments whose average is above 55000 (Civil at 45000.0 drops out). Question 9 keeps departments with more than three employees (Civil, with three, drops out).

2.4 Nested queries

A subquery is a SELECT inside another statement. A scalar subquery returns one value and can be compared with > or =. Question 10 compares each salary with the single overall average. An uncorrelated subquery does not refer to the outer query, so the engine can compute it once.

2.5 IN and NOT IN

IN (subquery) keeps an outer row when its value appears in the inner result. Question 13 keeps employees whose department is one of the departments that contain a Professor (CSE, Mechanical, ECE). NOT IN is the complement: question 14 keeps Civil, the only department with no Professor. If the subquery could return NULL, NOT IN would be unknown for every row and the result would be empty. Here department is NOT NULL, so that trap does not apply.

2.6 EXISTS and correlated subqueries

A correlated subquery mentions a column from the outer query, so it is re-evaluated for each outer row. EXISTS is true when the inner query returns at least one row; the selected columns do not matter, which is why question 15 selects the constant 1. It lists everyone who has a colleague in the same department on a higher salary — everyone except the highest-paid person in each department. Question 11 uses the same correlation to keep the row whose salary equals the department maximum. Question 16 compares each salary with the average of that employee's own department, which is a different number from the overall average used in question 10.

2.7 Second-highest salary

Question 12 nests two scalar queries. The innermost `MAX(salary)` is the top salary (92000). The middle query takes the maximum salary that is strictly less than that value, which is the next distinct salary (90000). The outer query returns the employee who earns it. This pattern still means “second-highest distinct salary” if two people share the top salary. On this table Priya Nair is the only employee at 90000.

3. Schema

One table, `employee`, holds six attributes. `sr_no` is the employee number (primary key). `department` is stored as a name so `GROUP BY` results can be read without a second lookup. `salary` is a positive monthly amount in rupees.

Attribute	Type	Constraint	Role
<code>sr_no</code>	INTEGER	PRIMARY KEY	Employee number
<code>employee_name</code>	TEXT	NOT NULL	Employee name
<code>job</code>	TEXT	NOT NULL	Job title
<code>department</code>	TEXT	NOT NULL	Department name
<code>salary</code>	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary (Rs)
<code>city</code>	TEXT	NOT NULL	Work city

4. Questions

Sixteen lab questions. Questions 1 and 2 build the table. Questions 3–6 are aggregates over every employee. Questions 7–9 group by department and, where asked, filter groups with `HAVING`. Questions 10–16 are nested queries: a comparison with the overall average, the highest salary in each department, the second-highest salary, `IN / NOT IN`, `EXISTS`, and a correlated department average.

- Create a table `employee` with the following attributes:
 - `sr_no`
 - `employee_name`
 - `job`
 - `department`
 - `salary`
 - `city`
- Insert 16 rows in the above mentioned table.
Four departments: CSE (5), Mechanical (4), ECE (4) and Civil (3). Civil has no Professor, so the `NOT IN` question is non-empty. The highest salary (92000) and the second-highest salary (90000) are each earned by one employee.
- Display the total number of employees from the `employee` table using `COUNT`.
- Display the total salary of all employees from the `employee` table using `SUM`.
- Display the average salary of all employees from the `employee` table using `AVG`.
- Display the maximum and minimum salary from the `employee` table using `MAX` and `MIN`.
- Display the number of employees, total salary, average salary, highest salary and lowest salary of each department using `COUNT`, `SUM`, `AVG`, `MAX`, `MIN` and `GROUP BY`.
- Display the department and average salary of departments whose average salary is greater than 55000 using `GROUP BY` and `HAVING`.

9. Display the department and the number of employees for departments that have more than 3 employees using GROUP BY and HAVING.
10. Display the employee name, department and salary of employees who earn more than the average salary, using a nested query.
11. Display the employee with the highest salary in each department using a subquery.
12. Display the employee name, department and salary of the employee who earns the second-highest salary, using a nested query.
13. Display employees who work in a department that has a Professor, using IN and a subquery.
14. Display employees who do not work in a department that has a Professor, using NOT IN and a subquery.
department is NOT NULL, so the subquery cannot return NULL. A NULL in a NOT IN list would make the predicate unknown for every row.
15. Display employees for whom there exists a higher-paid colleague in the same department, using EXISTS.
16. Display employees who earn more than the average salary of their own department, using a correlated subquery.
The comparison is strict. Rahul Das earns exactly the Civil average (45000), so this question does not list him.

5. Procedure

Drop employee if it already exists so the script can be re-run. Create the table, insert the 16 rows, then run each query in order. Grouped and nested results are ordered so the same script prints the same rows on every run. The SQL for each question:

Q. No.	SQL solution
1	<pre>CREATE TABLE employee (sr_no INTEGER PRIMARY KEY, employee_name TEXT NOT NULL, job TEXT NOT NULL, department TEXT NOT NULL, salary NUMERIC(10, 2) NOT NULL CHECK (salary > 0), city TEXT NOT NULL);</pre>
2	<pre>INSERT INTO employee (sr_no, employee_name, job, department, salary, city) VALUES (1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'), (2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'), (3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'), (4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'), (5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'), (6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'), (7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'), (8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'), (9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'), (10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'), (11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'), (12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'), (13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'), (14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'), (15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'), (16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur');</pre>
3	<pre>SELECT COUNT(*) AS employee_count FROM employee;</pre>
4	<pre>SELECT SUM(salary) AS total_salary FROM employee;</pre>

Q. No.	SQL solution
5	SELECT AVG(salary) AS average_salary FROM employee;
6	SELECT MAX(salary) AS maximum_salary, MIN(salary) AS minimum_salary FROM employee;
7	SELECT department, COUNT(*) AS employee_count, SUM(salary) AS total_salary, AVG(salary) AS average_salary, MAX(salary) AS highest_salary, MIN(salary) AS lowest_salary FROM employee GROUP BY department ORDER BY department;
8	SELECT department, AVG(salary) AS average_salary FROM employee GROUP BY department HAVING AVG(salary) > 55000 ORDER BY department;
9	SELECT department, COUNT(*) AS employee_count FROM employee GROUP BY department HAVING COUNT(*) > 3 ORDER BY department;
10	SELECT employee_name, department, salary FROM employee WHERE salary > (SELECT AVG(salary) FROM employee) ORDER BY salary DESC;
11	SELECT e.employee_name, e.department, e.salary FROM employee AS e WHERE e.salary = (SELECT MAX(c.salary) FROM employee AS c WHERE c.department = e.department) ORDER BY e.department;
12	SELECT employee_name, department, salary AS second_highest_salary FROM employee WHERE salary = (SELECT MAX(salary) FROM employee WHERE salary < (SELECT MAX(salary) FROM employee));
13	SELECT employee_name, job, department, salary FROM employee WHERE department IN (SELECT department FROM employee WHERE job = 'Professor') ORDER BY department, employee_name;
14	SELECT employee_name, job, department, salary FROM employee WHERE department NOT IN (SELECT department FROM employee WHERE job = 'Professor') ORDER BY department, employee_name;
15	SELECT e.employee_name, e.department, e.salary FROM employee AS e WHERE EXISTS (SELECT 1 FROM employee AS c WHERE c.department = e.department AND c.salary > e.salary) ORDER BY e.department, e.salary DESC;

Q. No.	SQL solution
16	<pre> SELECT e.employee_name, e.department, e.salary FROM employee AS e WHERE e.salary > (SELECT AVG(d.salary) FROM employee AS d WHERE d.department = e.department) ORDER BY e.department, e.salary DESC; </pre>

6. Source Code

SQLite script used in the lab compiler (30-09-2026/employee.sql). For the class SQL Server, run employee.sqlserver.sql.

```

-- =====
-- DBMS Lab · 30-09-2026
-- Employee table: CREATE, seed 16 rows, then aggregate functions
-- and nested queries.
-- SUM, AVG, MAX, MIN, COUNT, GROUP BY, HAVING
-- Subqueries: scalar, IN, NOT IN, EXISTS, correlated
-- SQLite (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employee.sqlserver.sql instead.
--
-- Lab questions
-- 1. Create a table employee with the following attributes:
--   sr no, employee name, job, department, salary, city
-- 2. Insert 16 rows in the above mentioned table
-- 3. Display the total number of employees using COUNT
-- 4. Display the total salary of all employees using SUM
-- 5. Display the average salary of all employees using AVG
-- 6. Display the maximum and minimum salary using MAX and MIN
-- 7. For each department, display COUNT, SUM, AVG, MAX and MIN
--   using GROUP BY
-- 8. Display departments whose average salary is greater than
--   55000 using GROUP BY and HAVING
-- 9. Display departments that have more than 3 employees
--   using GROUP BY and HAVING
-- 10. Display employees who earn more than the average salary
--   (nested query)
-- 11. Display the employee with the highest salary in each
--   department using a subquery
-- 12. Display the employee who earns the second-highest salary
--   using a nested query
-- 13. Display employees who work in a department that has a
--   Professor, using IN and a subquery
-- 14. Display employees who do not work in a department that
--   has a Professor, using NOT IN and a subquery
-- 15. Display employees for whom there EXISTS a higher-paid
--   colleague in the same department (correlated)
-- 16. Display employees who earn more than the average salary
--   of their own department (correlated subquery)
-- =====

DROP TABLE IF EXISTS employee;

-- -----
-- 1. Create a table employee with the following attributes:
--   sr no, employee name, job, department, salary, city
-- -----
CREATE TABLE employee (
  sr_no      INTEGER PRIMARY KEY,
  employee_name TEXT NOT NULL,
  job        TEXT NOT NULL,
  department TEXT NOT NULL,
  salary     NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
  city       TEXT NOT NULL
);

-- -----
-- 2. Insert 16 rows in the above mentioned table
--   • four departments: CSE (5), Mechanical (4), ECE (4), Civil (3)
--   • Civil has no Professor, so NOT IN is non-empty
--   • salaries vary so SUM, AVG, MAX, MIN and HAVING differ
--   • the overall maximum (92000) and second maximum (90000)
--   are each earned by exactly one employee
-- -----
INSERT INTO employee
(sr_no, employee_name, job, department, salary, city)
VALUES
(1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'),
(2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'),
(3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'),
(4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'),
(5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'),
(6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'),

```

```

(7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'),
(8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'),
(9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'),
(10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'),
(11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'),
(12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'),
(13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'),
(14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'),
(15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'),
(16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur');

```

```

-- -----
-- 3. Display the total number of employees from the employee
-- table using COUNT
-- -----

```

```

-- Total number of employees
SELECT COUNT(*) AS employee_count
FROM employee;

```

```

-- -----
-- 4. Display the total salary of all employees from the
-- employee table using SUM
-- -----

```

```

-- Total salary of all employees
SELECT SUM(salary) AS total_salary
FROM employee;

```

```

-- -----
-- 5. Display the average salary of all employees from the
-- employee table using AVG
-- -----

```

```

-- Average salary of all employees
SELECT AVG(salary) AS average_salary
FROM employee;

```

```

-- -----
-- 6. Display the maximum and minimum salary from the employee
-- table using MAX and MIN
-- -----

```

```

-- Maximum and minimum salary
SELECT MAX(salary) AS maximum_salary,
       MIN(salary) AS minimum_salary
FROM employee;

```

```

-- -----
-- 7. Display the number of employees, total salary, average
-- salary, highest salary and lowest salary of each
-- department using COUNT, SUM, AVG, MAX, MIN and GROUP BY
-- -----

```

```

-- Aggregates for each department
SELECT department,
       COUNT(*) AS employee_count,
       SUM(salary) AS total_salary,
       AVG(salary) AS average_salary,
       MAX(salary) AS highest_salary,
       MIN(salary) AS lowest_salary
FROM employee
GROUP BY department
ORDER BY department;

```

```

-- -----
-- 8. Display the department and average salary of departments
-- whose average salary is greater than 55000 using
-- GROUP BY and HAVING
-- -----

```

```

-- Departments with average salary above 55000
SELECT department,
       AVG(salary) AS average_salary
FROM employee
GROUP BY department
HAVING AVG(salary) > 55000
ORDER BY department;

```

```

-- -----
-- 9. Display the department and the number of employees for
-- departments that have more than 3 employees using
-- GROUP BY and HAVING
-- -----

```

```

-- Departments with more than 3 employees
SELECT department,
       COUNT(*) AS employee_count
FROM employee
GROUP BY department
HAVING COUNT(*) > 3
ORDER BY department;

```

```

-- -----
-- 10. Display the employee name, department and salary of
-- employees who earn more than the average salary,
-- using a nested query
-- -----

```

```

-- Employees earning more than the average salary

```

```

SELECT employee_name, department, salary
FROM employee
WHERE salary > (SELECT AVG(salary) FROM employee)
ORDER BY salary DESC;

-- -----
-- 11. Display the employee with the highest salary in each
--     department using a subquery
-- -----
-- Highest salary in each department
SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE e.salary = (
    SELECT MAX(c.salary)
    FROM employee AS c
    WHERE c.department = e.department
)
ORDER BY e.department;

-- -----
-- 12. Display the employee name, department and salary of the
--     employee who earns the second-highest salary, using a
--     nested query
-- -----
-- Second-highest salary
SELECT employee_name, department, salary AS second_highest_salary
FROM employee
WHERE salary = (
    SELECT MAX(salary)
    FROM employee
    WHERE salary < (SELECT MAX(salary) FROM employee)
);

-- -----
-- 13. Display employees who work in a department that has a
--     Professor, using IN and a subquery
-- -----
-- Employees in a department that has a Professor
SELECT employee_name, job, department, salary
FROM employee
WHERE department IN (
    SELECT department
    FROM employee
    WHERE job = 'Professor'
)
ORDER BY department, employee_name;

-- -----
-- 14. Display employees who do not work in a department that
--     has a Professor, using NOT IN and a subquery
-- -----
-- Employees not in a department that has a Professor
SELECT employee_name, job, department, salary
FROM employee
WHERE department NOT IN (
    SELECT department
    FROM employee
    WHERE job = 'Professor'
)
ORDER BY department, employee_name;

-- -----
-- 15. Display employees for whom there EXISTS a higher-paid
--     colleague in the same department (correlated subquery)
-- -----
-- Employees with a higher-paid colleague
SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE EXISTS (
    SELECT 1
    FROM employee AS c
    WHERE c.department = e.department
        AND c.salary > e.salary
)
ORDER BY e.department, e.salary DESC;

-- -----
-- 16. Display employees who earn more than the average salary
--     of their own department (correlated subquery)
-- -----
-- Employees above their department average
SELECT e.employee_name, e.department, e.salary
FROM employee AS e
WHERE e.salary > (
    SELECT AVG(d.salary)
    FROM employee AS d
    WHERE d.department = e.department
)
ORDER BY e.department, e.salary DESC;

```

7. Output

The script was executed in SQLite in memory. CREATE prints a status line and INSERT prints the row count. Each SELECT prints the result set that SQLite returned. Whole-number salaries print as integers; AVG prints a real (56125.0, 59200.0, 58750.0, and so on).

```
SQL> DROP TABLE IF EXISTS employee;
-- OK

SQL> CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,
    job            TEXT NOT NULL,
    department     TEXT NOT NULL,
    salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    city          TEXT NOT NULL
);
-- OK

SQL> INSERT INTO employee
(sr_no, employee_name, job, department, salary, city)
VALUES
(1, 'Rajesh Kumar', 'Professor', 'CSE', 92000.00, 'Delhi'),
(2, 'Ananya Sharma', 'Associate Professor', 'CSE', 78000.00, 'Bangalore'),
(3, 'Fatima Khan', 'Assistant Professor', 'CSE', 56000.00, 'Bangalore'),
(4, 'Karan Iyer', 'Lab Instructor', 'CSE', 42000.00, 'Mumbai'),
(5, 'Dev Patel', 'Teaching Assistant', 'CSE', 28000.00, 'Ahmedabad'),
(6, 'Vikram Singh', 'Professor', 'Mechanical', 88000.00, 'Hyderabad'),
(7, 'Rohan Mehta', 'Associate Professor', 'Mechanical', 64000.00, 'Mumbai'),
(8, 'Arjun Patel', 'Assistant Professor', 'Mechanical', 48000.00, 'Pune'),
(9, 'Mohit Jain', 'Lab Instructor', 'Mechanical', 32000.00, 'Jaipur'),
(10, 'Priya Nair', 'Professor', 'ECE', 90000.00, 'Chennai'),
(11, 'Neha Gupta', 'Associate Professor', 'ECE', 58000.00, 'Delhi'),
(12, 'Meera Joshi', 'Assistant Professor', 'ECE', 51000.00, 'Chennai'),
(13, 'Kavya Menon', 'Lab Instructor', 'ECE', 36000.00, 'Kochi'),
(14, 'Sneha Reddy', 'Associate Professor', 'Civil', 60000.00, 'Hyderabad'),
(15, 'Rahul Das', 'Assistant Professor', 'Civil', 45000.00, 'Kolkata'),
(16, 'Tushar Rao', 'Lab Instructor', 'Civil', 30000.00, 'Nagpur');
-- 16 row(s) inserted

SQL> SELECT COUNT(*) AS employee_count
FROM employee;
employee_count
-----
16

SQL> SELECT SUM(salary) AS total_salary
FROM employee;
total_salary
-----
898000

SQL> SELECT AVG(salary) AS average_salary
FROM employee;
average_salary
-----
56125.0

SQL> SELECT MAX(salary) AS maximum_salary,
MIN(salary) AS minimum_salary
FROM employee;
maximum_salary  minimum_salary
-----
92000           28000

SQL> SELECT department,
COUNT(*) AS employee_count,
SUM(salary) AS total_salary,
AVG(salary) AS average_salary,
MAX(salary) AS highest_salary,
MIN(salary) AS lowest_salary
FROM employee
GROUP BY department
ORDER BY department;
department  employee_count  total_salary  average_salary  highest_salary  lowest_salary
-----
CSE         5              296000      59200.0        92000          28000
Civil       3              135000      45000.0        60000          30000
ECE         4              235000      58750.0        90000          36000
Mechanical  4              232000      58000.0        88000          32000

SQL> SELECT department,
AVG(salary) AS average_salary
FROM employee
GROUP BY department
HAVING AVG(salary) > 55000
ORDER BY department;
department  average_salary
-----
CSE         59200.0
ECE         58750.0
Mechanical  58000.0

SQL> SELECT department,
COUNT(*) AS employee_count
FROM employee
GROUP BY department
HAVING COUNT(*) > 3
ORDER BY department;
department  employee_count
-----
CSE         5
ECE         4
```

Mechanical 4

```
SQL> SELECT employee_name, department, salary
      FROM employee
      WHERE salary > (SELECT AVG(salary) FROM employee)
      ORDER BY salary DESC;
employee_name  department  salary
-----
Rajesh Kumar   CSE         92000
Priya Nair     ECE         90000
Vikram Singh   Mechanical  88000
Ananya Sharma  CSE         78000
Rohan Mehta    Mechanical  64000
Sneha Reddy    Civil       60000
Neha Gupta     ECE         58000
```

```
SQL> SELECT e.employee_name, e.department, e.salary
      FROM employee AS e
      WHERE e.salary = (
        SELECT MAX(c.salary)
        FROM employee AS c
        WHERE c.department = e.department
      )
      ORDER BY e.department;
employee_name  department  salary
-----
Rajesh Kumar   CSE         92000
Sneha Reddy    Civil       60000
Priya Nair     ECE         90000
Vikram Singh   Mechanical  88000
```

```
SQL> SELECT employee_name, department, salary AS second_highest_salary
      FROM employee
      WHERE salary = (
        SELECT MAX(salary)
        FROM employee
        WHERE salary < (SELECT MAX(salary) FROM employee)
      );
employee_name  department  second_highest_salary
-----
Priya Nair     ECE         90000
```

```
SQL> SELECT employee_name, job, department, salary
      FROM employee
      WHERE department IN (
        SELECT department
        FROM employee
        WHERE job = 'Professor'
      )
      ORDER BY department, employee_name;
employee_name  job              department  salary
-----
Ananya Sharma  Associate Professor CSE         78000
Dev Patel      Teaching Assistant CSE         28000
Fatima Khan    Assistant Professor CSE         56000
Karan Iyer     Lab Instructor     CSE         42000
Rajesh Kumar   Professor          CSE         92000
Kavya Menon    Lab Instructor     ECE         36000
Meera Joshi    Assistant Professor ECE         51000
Neha Gupta     Associate Professor ECE         58000
Priya Nair     Professor          ECE         90000
Arjun Patel    Assistant Professor Mechanical  48000
Mohit Jain     Lab Instructor     Mechanical  32000
Rohan Mehta    Associate Professor Mechanical  64000
Vikram Singh   Professor          Mechanical  88000
```

```
SQL> SELECT employee_name, job, department, salary
      FROM employee
      WHERE department NOT IN (
        SELECT department
        FROM employee
        WHERE job = 'Professor'
      )
      ORDER BY department, employee_name;
employee_name  job              department  salary
-----
Rahul Das     Assistant Professor Civil       45000
Sneha Reddy    Associate Professor Civil       60000
Tushar Rao     Lab Instructor    Civil       30000
```

```
SQL> SELECT e.employee_name, e.department, e.salary
      FROM employee AS e
      WHERE EXISTS (
        SELECT 1
        FROM employee AS c
        WHERE c.department = e.department
        AND c.salary > e.salary
      )
      ORDER BY e.department, e.salary DESC;
employee_name  department  salary
-----
Ananya Sharma  CSE         78000
Fatima Khan    CSE         56000
Karan Iyer     CSE         42000
Dev Patel      CSE         28000
Rahul Das     Civil       45000
Tushar Rao     Civil       30000
Neha Gupta     ECE         58000
Meera Joshi    ECE         51000
Kavya Menon    ECE         36000
Rohan Mehta    Mechanical  64000
Arjun Patel    Mechanical  48000
Mohit Jain     Mechanical  32000
```

```
SQL> SELECT e.employee_name, e.department, e.salary
```

```

FROM employee AS e
WHERE e.salary > (
    SELECT AVG(d.salary)
    FROM employee AS d
    WHERE d.department = e.department
)
ORDER BY e.department, e.salary DESC;
employee_name department salary
-----
Rajesh Kumar    CSE          92000
Ananya Sharma   CSE          78000
Sneha Reddy     Civil        60000
Priya Nair      ECE          90000
Vikram Singh    Mechanical   88000
Rohan Mehta     Mechanical   64000

```

8. Results

Sixteen employees load successfully. Every question returns a non-empty result on this seed. The figures below are the row counts and values from the SQLite run in section 7.

Q. No.	Rows	What the result shows
1	—	Table employee created with 6 attributes
2	16	16 rows inserted across CSE, Mechanical, ECE and Civil
3	1	COUNT(*) = 16 employees
4	1	SUM(salary) = 898000
5	1	AVG(salary) = 56125.0
6	1	MAX = 92000 and MIN = 28000
7	4	COUNT, SUM, AVG, MAX and MIN for each of the 4 departments
8	3	CSE (59200.0), ECE (58750.0), Mechanical (58000.0); Civil excluded
9	3	CSE (5), ECE (4), Mechanical (4); Civil has only 3
10	7	Names earning more than the overall average 56125.0
11	4	Rajesh Kumar, Sneha Reddy, Priya Nair, Vikram Singh
12	1	Second-highest salary: Priya Nair, ECE, 90000
13	13	IN: employees in CSE, Mechanical or ECE (a Professor exists)
14	3	NOT IN: Rahul Das, Sneha Reddy and Tushar Rao (Civil)
15	12	EXISTS: everyone except the 4 department-wise top salaries
16	6	Correlated: salary greater than that department's own average

Department totals from question 7 add up to the overall total: 296000 + 135000 + 235000 + 232000 = 898000, and 5 + 3 + 4 + 4 = 16. Question 10 returns seven employees above the company average of 56125.0. Question 16 returns six employees above their own department average. Neha Gupta (ECE, 58000) is in the first list only: 58000 is above 56125.0, but the ECE average is 58750.0. Fatima Khan (56000) is below both the company average and the CSE average (59200.0). Rahul Das earns exactly the Civil average (45000), so question 16 excludes him, and 45000 is also below the company average.

Question 12 returns one row because 90000 occurs once. Question 15 returns 12 rows: 16 employees minus the four people who have no higher-paid colleague in their department, the same four names question 11 returns.

9. Conclusion

Aggregate functions collapse many rows into one value, and GROUP BY repeats that collapse once per group. HAVING then keeps or drops whole groups. A nested query uses one result as the input of another: a scalar subquery for a comparison, IN / NOT IN for membership, and EXISTS for a yes-or-no test. When the inner query refers to the outer row it is correlated, so each employee can be compared with their own department rather than with the whole company. Together these are the usual tools for summary questions and for questions of the form “more than the average” or “the highest in each group”.